Executive Project Report: Transitioning from Naive RAG to Evaluation-Driven Engineering

1. Strategic Vision and Mission

The current AI landscape is characterized by a significant industry gap: the distance between "naive RAG" prototypes and production-grade engineering. While standard implementations focus on a "Document-to-Chat" interface—often relying on fragile "vibe checks" and hope—enterprise-ready systems require a "Document-to-Evaluation" architecture.This project shifts the focus from subjective fluency to objective, empirical verification. By treating AI development as a rigorous engineering discipline, we replace experimental prompting with a system designed for reliability, observability, and systematic optimization. Our goal is to move beyond simple chatbots toward a sophisticated evaluation platform that provides a verifiable feedback loop for professional AI teams.**Core Mission** To architect a production-grade AI evaluation platform that systematically measures and optimizes RAG performance, targeting a minimum **90%+ Faithfulness score** via the Ragas framework before any configuration is promoted to production.

2. Comparative Analysis: Standard vs. Production Workflows

To distinguish this project from entry-level implementations, the following table contrasts the "Upload-Chat-Hope" anti-pattern with our data-driven engineering cycle.| Feature | Standard Portfolio Projects | Production AI Development Platform || ------ | ------ | ------ || **Workflow** | Upload -> Chat -> Answer | Upload -> Retrieval -> Answer -> Evaluation -> Metrics -> Experimentation || **Focus** | Basic functionality; "chat-like" feel | Reliability, observability, and systematic optimization || **Philosophy** | Success measured by a single coherent response (Subjective) | Success measured by statistical accuracy across thousands of queries (Objective) || **Outcome** | Standalone "toy" application | Enterprise-grade data platform |

3. The Philosophy of Evaluation-Driven Development (EDD)

Evaluation-Driven Development (EDD) mandates that AI systems be constructed with the same rigor as mission-critical software. This requires high standards for observability and structural integrity.

- **Design Transparency:** The platform enforces a clean separation between the "AI Answer," the "Evaluation of the Answer," and the specific source chunks used for context. Exposing this lineage proves to stakeholders that the output is grounded in truth, not LLM hallucination.
- **Zero-Orphan Database Philosophy:** To maintain system integrity and manage cloud overhead, we implement strict ON DELETE CASCADE rules. When a document is expunged, PostgreSQL atomically cleans the vector space. This prevents stale data from poisoning future retrievals and **eliminates hidden cloud database storage fees** associated with orphaned embeddings.

4. Technical Architecture: Data Tier and Vector Integrity

The data tier is built on Supabase, utilizing PostgreSQL with the pgvector extension. We utilize the text-embedding-3-small model to generate high-efficiency, high-accuracy embeddings.

- **Schema Definition:** The public.document_chunks table is configured with a vector(1536) embedding column, explicitly mapped to the text-embedding-3-small output.
- **Vector Validation Metric:** Structural health is confirmed via the vector_dims(embedding) function. The system enforces a validated metric of exactly **1536 dimensions** to prevent indexing failures.
- **Security Architecture:**
- **Row Level Security (RLS):** Enabled on all user-facing tables to ensure multi-tenant isolation (auth.uid() = user_id).
- **Administrative Bypass:** Background ingestion and evaluation tasks utilize a privileged supabaseAdmin client via the SUPABASE_SERVICE_ROLE_KEY to operate safely across data boundaries without compromising client-side security.

5. The Unified Ingestion Engine (Phases 3 & 4)

We have rejected decoupled, multi-hop asynchronous architectures in favor of a **Unified Ingestion Route (/api/process)** . This route executes as a single atomic transaction window:

```
Fetch Storage File ⟶ LlamaIndex Token Chunking ⟶ OpenAI Vectorization (text-embedding-3-small) ⟶ Atomic DB Insert
```

Senior Execution Guardrails:

- **Orchestration Library Lock:** We utilize **LlamaIndex node parsers** (specifically SentenceSplitter from @llamaindex/core) to ensure uniform chunk mapping across vector maps, avoiding the inconsistency of custom string slicing.
- **The Array Batch Limit:** To prevent **database pool connection lag** and avoid **OpenAI 429 Rate Limit crashes** , the engine processes embeddings in deterministic batches of exactly **20 elements** per iteration.
- **Timeout Defense:** To protect intensive extractions from serverless execution limits, the route handler is declared with export const runtime = "nodejs", utilizing a dedicated long-running execution layer.

6. Programmatic Retrieval and Matching Logic

Phase 5 pushes vector searching into a compiled PostgreSQL stored procedure. This maximizes processing speed and minimizes cold-start query latency.**Cosine Similarity Engine:** The system calculates matches using the following mathematical logic, utilizing the pgvector distance operator (<=>):$$\text{Similarity} = 1 - (\text{document\_chunks.embedding} \Leftrightarrow \text{query\_embedding})$$*Note: The <=> operator represents the cosine distance; subtracting it from 1 provides the similarity score required for ranking.*While user-facing retrieval routes are locked tight with RLS to prevent horizontal data exposure, the supabaseAdmin client handles the high-privilege processing required for background evaluation and metric generation.

7. The 10-Phase Roadmap and Current Project Status

Progress is strictly gated behind infrastructure verification.

1. **Project Setup:** Next.js skeleton with live Supabase Auth and verified middleware. ✅ **FUNCTIONAL**

2. **File Management:** Ingestion hooks wired to Storage with metadata logging and UI feedback. ✅ **COMPLETE**
3. **Processing & Text Extraction:** Unified data pipelines turning inputs into atomic text streams. ✅ **COMPLETE**
4. **Embeddings:** Generating 1536-dimensional coordinates via text-embedding-3-small. ✅ **COMPLETE**
5. **Retrieval:** Compiling database RPC matches and building secure server-side routes. 🏗️ **ACTIVE**
6. **AI Answers:** Grounded prompt execution parsing context arrays into user answers. ⏳ **PENDING**
7. **Evaluation:** Automated output parsing and scoring through the Ragas framework. ⏳ **PENDING**
8. **Experimentation:** System dashboards evaluating multi-tenant chunk and prompt configurations. ⏳ **PENDING**
9. **Observability:** Full-stack tracing via hierarchical parent-child spans in Langfuse. ⏳ **PENDING**
10. **Portfolio Polish:** Compiling case studies showing metrics, cost maps, and architecture charts. ⏳ **PENDING**

8. System Credentials and Security Hydration

The infrastructure is secured via environment variable gating and multi-tenant isolation boundaries.**Environment Credentials Status**| Variable | Status | Purpose || ------ | ------ | ------ || NEXT_PUBLIC_SUPABASE_URL | **ACTIVE** | Core Database/Storage URL || NEXT_PUBLIC_SUPABASE_ANON_KEY | **ACTIVE** | Client-side access || SUPABASE_SERVICE_ROLE_KEY | **ACTIVE** | Server-side ingestion bypass (Phase 3+) || OPENAI_API_KEY | **ACTIVE** | text-embedding-3-small and Inference || LANGFUSE_KEYS | **PENDING** | Observability (Active in Phase 9) |

Storage and Isolation Boundaries: Security is enforced through three primary Supabase storage policies (Upload, Read, and Delete). Data is strictly isolated by auth.uid(), ensuring that users can only interact with files stored within their unique folder ({user_id}/{timestamp}_{name}). This provides a robust boundary against horizontal data exposure and unauthorized access.